

BUỔI 2. LINUX SERVER VÀ TRIỂN KHAI ỨNG DỤNG MERN

Môi trường: GitHub Codespaces

Database: MongoDB Atlas Free Tier

Ứng dụng: Quản lý sinh viên

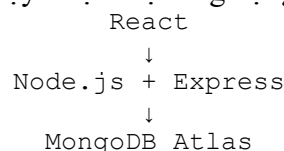
Mục tiêu: Linux Server và triển khai ứng dụng MERN

Sau khi hoàn thành buổi thực hành, sinh viên có khả năng:

1. **Làm việc với Linux Server trên môi trường Cloud** thông qua GitHub Codespaces và thực hiện các thao tác cơ bản trên Linux bằng Terminal.
2. **Kiểm tra và quản lý tài nguyên Linux Server**, bao gồm User, Permission, Process, Port và các thông tin cơ bản của hệ điều hành.
3. **Cài đặt và sử dụng Node.js trên Linux Server** để xây dựng và chạy một ứng dụng Web.
4. **Xây dựng Backend bằng Node.js và Express**, tạo và kiểm thử các REST API trên Linux Server.
5. **Kết nối ứng dụng Backend với MongoDB Atlas**, qua đó làm quen với mô hình sử dụng Cloud Database.
6. **Xây dựng giao diện React** và kết nối React với REST API của Backend.
7. **Thực hiện các chức năng CRUD** (Thêm, Xem, Sửa, Xóa) dữ liệu sinh viên thông qua ứng dụng MERN.
8. **Quản lý thông tin cấu hình bằng biến môi trường**, đồng thời bảo vệ thông tin kết nối Database bằng `.env` và `.gitignore`.
9. **Sử dụng Git và GitHub để quản lý source code**, commit và push kết quả thực hành lên Repository.
10. **Hiểu được quy trình triển khai một ứng dụng Web trên Cloud**, từ Linux Server → Backend → Frontend → Cloud Database.

Kết quả cần đạt

Cuối buổi, sinh viên xây dựng và chạy được một ứng dụng MERN theo mô hình:



và có thể giải thích được vai trò của Linux Server, Process, Port, Backend, Frontend và Cloud Database trong một ứng dụng Web thực tế.

PHẦN 1. KHỞI TẠO VÀ KIỂM TRA LINUX SERVER

Câu 1. Mở Repository `cloud-lab` đã tạo ở Buổi 1 và tạo một GitHub Codespace từ branch `main`. (Gợi ý: Chọn Code → Codespaces → Create codespace on main)

Câu 2. Mở Terminal trong GitHub Codespaces để thực hiện các lệnh Linux. (Gợi ý: Chọn Terminal → New Terminal)

Câu 3. Kiểm tra thư mục hiện tại. (Gợi ý lệnh: `pwd`)

Câu 4. Liệt kê các file và thư mục, bao gồm cả file ẩn. (Gợi ý lệnh: `ls -la`)

Câu 5. Kiểm tra username hiện tại. (Gợi ý lệnh: `whoami`)

Câu 6. Kiểm tra thông tin hệ điều hành Linux. (Gợi ý lệnh: `cat /etc/os-release`)

Câu 7. Kiểm tra phiên bản Kernel Linux. (Gợi ý lệnh: `uname -a`)

Câu 8. Kiểm tra dung lượng RAM và ổ đĩa của Linux Server. (Gợi ý lệnh: `free -h` và `df -h`)

Câu 9. Kiểm tra phiên bản Node.js và npm trên Linux Server. (Gợi ý lệnh: `node --version` và `npm --version`)

Mục tiêu phần này: Sinh viên xác định được mình đang làm việc trên một Linux Server và biết cách kiểm tra các thông tin cơ bản của Server.

PHẦN 2. USER, PERMISSION VÀ PROCESS

Câu 10. Kiểm tra thông tin User và Group hiện tại. (Gợi ý lệnh: `id`)

Câu 11. Tạo thư mục `permission-test` và tạo file `config.txt` bên trong. (Gợi ý lệnh: `mkdir permission-test && touch permission-test/config.txt`)

Câu 12. Kiểm tra Owner, Group và Permission của file `config.txt`. (Gợi ý lệnh: `ls -l permission-test`)

Câu 13. Thiết lập Permission 600 cho file `config.txt`. (Gợi ý lệnh: `chmod 600 permission-test/config.txt`)

Câu 14. Kiểm tra lại Permission của file sau khi thay đổi. (Gợi ý lệnh: `ls -l permission-test/config.txt`)

Câu 15. Hiển thị các Process đang chạy trên Linux Server. (Gợi ý lệnh: `ps aux`)

Câu 16. Kiểm tra các Port đang ở trạng thái Listen. (Gợi ý lệnh: `ss -lnt`)

Mục tiêu: Sinh viên hiểu được ba khái niệm sẽ sử dụng xuyên suốt môn học: **User – Process – Port**.

PHẦN 3. NODE.JS VÀ EXPRESS

3.1. Tạo Backend

Câu 17. Tạo thư mục `mern-demo` và di chuyển vào thư mục này. (Gợi ý lệnh: `mkdir mern-demo && cd mern-demo`)

Câu 18. Khởi tạo Node.js Project. (Gợi ý lệnh: `npm init -y`)

Câu 19. Cài đặt các package `express`, `cors` và `dotenv`. (Gợi ý lệnh: `npm install express cors dotenv`)

Câu 20. Tạo thư mục `server` và file `server.js`. (Gợi ý lệnh: `mkdir server && touch server/server.js`)

Câu 21. Xây dựng Express Server chạy trên port 5000. (Gợi ý: nhập trực tiếp trong file `server.js`, Sử dụng package `express`)

Câu 22. Tạo API GET `/api/hello` trả về thông báo xác nhận Backend đang hoạt động. (Gợi ý: tạo api này trong `server.js`)

Câu 23. Cấu hình lệnh khởi động Backend trong file `package.json`. (Gợi ý: Thêm script `"start": "node server/server.js"` và sử dụng lệnh `npm start`)

Câu 24. Kiểm tra Process Node.js đang chạy trên Linux Server. (Gợi ý: Sử dụng lệnh `ps aux | grep node`)

Câu 25. Kiểm tra port 5000 đang được Node.js sử dụng. (Gợi ý: Sử dụng lệnh `ss -lntp | grep 5000`)

Câu 26. Sử dụng `curl` để kiểm tra API Backend. (Gợi ý: Sử dụng lệnh `curl http://localhost:5000/api/hello`)

PHẦN 4. KẾT NỐI MONGODB ATLAS

- Câu 27.** Tạo một MongoDB Atlas Database Deployment miễn phí. (Gợi ý: Sử dụng Free Tier)
- Câu 28.** Tạo Database `cloud_lab` và Collection `students`. (Gợi ý: Thực hiện trong MongoDB Atlas)
- Câu 29.** Tạo Database User để Backend có thể kết nối MongoDB Atlas. (Gợi ý: Sử dụng chức năng Database Access)
- Câu 30.** Lấy MongoDB Connection String dành cho Node.js. (Gợi ý: Chọn Connect → Drivers → Node.js)
- Câu 31.** Cài đặt thư viện Mongoose. (Gợi ý lệnh: `npm install mongoose`)
- Câu 32.** Tạo file `.env` để lưu `MONGODB_URI` và `PORT`. (Gợi ý: Khai báo `MONGODB_URI=mongodb+srv://... và PORT=5000`)
- Câu 33.** Kết nối Express Backend với MongoDB Atlas. (Gợi ý: Sử dụng `mongoose.connect(process.env.MONGODB_URI)`)
- Câu 34.** Khởi động Backend và kiểm tra log để xác nhận kết nối MongoDB Atlas thành công. (Gợi ý lệnh: `node server/server.js`)
-

PHẦN 5. XÂY DỰNG REST API

Sinh viên xây dựng API quản lý sinh viên.

- Câu 35.** Tạo Model `Student` gồm `studentId`, `name` và `email`. (Gợi ý: Sử dụng Mongoose Schema)
- Câu 36.** Xây dựng API `GET /api/students` để lấy danh sách sinh viên. (Gợi ý: Sử dụng `Student.find()`)
- Câu 37.** Xây dựng API `POST /api/students` để thêm sinh viên. (Gợi ý: Sử dụng `Student.create()` và `req.body`)
- Câu 38.** Xây dựng API `PUT /api/students/:id` để cập nhật sinh viên. (Gợi ý: Sử dụng `Student.findByIdAndUpdate()`)
- Câu 39.** Xây dựng API `DELETE /api/students/:id` để xóa sinh viên. (Gợi ý: Sử dụng `Student.findByIdAndDelete()`)
- Câu 40.** Sử dụng `curl` để kiểm tra API `GET /api/students`. (Gợi ý lệnh: `curl http://localhost:5000/api/students`)
- Câu 41.** Sử dụng `curl` để gửi Request `POST` tạo một sinh viên mới. (Gợi ý: Sử dụng HTTP Method `POST` và dữ liệu `JSON`)
-

PHẦN 6. XÂY DỰNG REACT FRONTEND

- Câu 42.** Tạo React Application bằng Vite trong thư mục `client`. (Gợi ý lệnh: `npm create vite@latest client -- --template react`)
- Câu 43.** Cài đặt các package của React Application. (Gợi ý lệnh: `cd client && npm install`)
- Câu 44.** Khởi động React Development Server và cho phép truy cập từ bên ngoài. (Gợi ý lệnh: `npm run dev -- --host 0.0.0.0`)
- Câu 45.** Kiểm tra port mà React Development Server đang sử dụng. (Gợi ý lệnh: `ss -lntp`)
- Câu 46.** Mở port React thông qua GitHub Codespaces và truy cập ứng dụng bằng trình duyệt. (Gợi ý: Mở port 5173 trong mục PORTS)
- Câu 47.** Xây dựng giao diện hiển thị danh sách sinh viên từ Backend API. (Gợi ý: Sử dụng `fetch()` hoặc `Axios` gọi `GET /api/students`)
- Câu 48.** Tạo Form nhập MSSV, Họ tên và Email. (Gợi ý: Sử dụng React State và các thành phần `<input>`)

Câu 49. Gửi dữ liệu từ React đến API `POST /api/students`. (Gợi ý: Sử dụng `fetch()` hoặc `Axios` với `HTTP Method POST`)

Câu 50. Kiểm tra dữ liệu sinh viên vừa thêm trong MongoDB Atlas. (Gợi ý: Mở `Collection students` và kiểm tra `Document`)

PHẦN 7. BIẾN MÔI TRƯỜNG VÀ GIT

Câu 51. Thêm file `.env` vào `.gitignore`. (Gợi ý: Thêm dòng `.env` vào file `.gitignore`)

Câu 52. Tạo file `.env.example` chỉ chứa tên các biến môi trường và không chứa thông tin kết nối thật. (Gợi ý: Khai báo `PORT=` và `MONGODB_URI=`)

Câu 53. Kiểm tra Git để đảm bảo file `.env` không được đưa vào Staging Area. (Gợi ý lệnh: `git status`)

Câu 54. Kiểm tra toàn bộ source code trước khi commit. (Gợi ý lệnh: `git status`)

Câu 55. Đưa source code vào Staging Area. (Gợi ý lệnh: `git add .`)

Câu 56. Tạo commit cho kết quả thực hành Buổi 2. (Gợi ý lệnh: `git commit -m "Complete Lab 02 MERN"`)

Câu 57. Push source code lên GitHub. (Gợi ý lệnh: `git push`)

PHẦN 8. BÀI TẬP TỔNG HỢP

Câu 58. Kiểm tra Linux Server và xác nhận Node.js Backend đang hoạt động. (Gợi ý: Kiểm tra bằng `ps aux | grep node` và `ss -lntp | grep 5000`)

Câu 59. Truy cập React Frontend trên trình duyệt và hiển thị danh sách sinh viên từ MongoDB Atlas. (Gợi ý: `React` → `API` → `Express` → `MongoDB Atlas`)

Câu 60. Thêm ít nhất 3 sinh viên từ giao diện React và kiểm tra dữ liệu trên MongoDB Atlas. (Gợi ý: Sử dụng `Form nhập dữ liệu` và `API POST`)

Câu 61. Thực hiện cập nhật một sinh viên thông qua ứng dụng. (Gợi ý: Sử dụng `API PUT /api/students/:id`)

Câu 62. Thực hiện xóa một sinh viên thông qua ứng dụng. (Gợi ý: Sử dụng `API DELETE /api/students/:id`)

Câu 63. Kiểm tra lại danh sách sinh viên sau khi thêm, sửa và xóa dữ liệu. (Gợi ý: Gọi lại `API GET /api/students`)

Câu 64. Kiểm tra Repository trên GitHub và xác nhận source code MERN đã được Push thành công. (Gợi ý: Kiểm tra thư mục `client`, `server` và các file cấu hình)